

03 SaborExpress Propuesta Tecnica

SABOREXPRESS — PROYECTO DE SOFTWARE

Yerson Stiven Cuellar Rubiano

Ficha de formación: 3145555

Expediente técnico y académico — APA 7

14 de octubre de 2026

Fuente documental: repositorios SaborExpressBack, SaborExpressWeb y SaborExpressMovil.

Las pruebas no ejecutadas se identifican como pendientes.

Propuesta técnica y plan de solución

Proyecto: SaborExpress | **Autor / integrante:** Yerson Stiven Cuellar Rubiano | **Ficha:** 3145555 | **Fecha:** 14 de octubre de 2026 | **Versión documental:** 4.0

Estado de la evidencia: inspección documental y estática del código. Las pruebas de ejecución y los datos institucionales no aportados no se presumen realizados.

1. Resumen ejecutivo

Se propone mantener y estabilizar una plataforma modular compuesta por una API .NET, un portal de administración Next.js y una aplicación Expo para usuarios/roles de operación. La decisión prioritaria no es añadir microservicios, sino asegurar integridad de transacciones, autorización por sede, pruebas de pagos y despliegue reproducible.

2. Objetivo, alcance y entregables

El objetivo es documentar y validar los componentes ya presentes, integrarlos mediante un contrato OpenAPI verificable y asegurar una puesta en marcha comprobable. Entregables: SRS trazable, diagramas, manual técnico, manual de instalación, manual de usuario, catálogo de rutas, diccionario EF, matriz de pruebas, plan de mejora y artículo científico.

3. Alternativas de arquitectura

Opción	Ventaja	Riesgo/condición	Decisión
Monolito modular .NET	Menor complejidad de despliegue; actual	Exige límites de dominio claros	Mantener
Microservicios por dominio	Escalado y autonomía independientes	Costo operativo y consistencia distribuidos	No afirmar ni migrar sin evidencia
Supabase como backend único	Servicios integrados	No coincide con código actual	Descartada como descripción actual
Dos frontends API compartida	Reuso de contratos de negocio	Versionado y permisos	Mantener y probar

4. Plan de implementación y responsables

Fase	Actividades	Criterio de cierre
P1 – saneamiento	Congelar commit, corregir README, inventariar secretos y dependencias	Inventario y .env.example publicados
P2 – integridad	Migraciones reproducibles, pruebas CRUD por proveedor objetivo	Test y restauración pasan
P3 – autorizaciones	Matriz rol/recurso/sede; pruebas negativas	0 accesos indebidos en matriz
P4 – pagos	Sandbox Wompi, validación de firmas y duplicados	Conciliación sin doble cargo
P5 – operación	Ensayos E2E y monitoreo; backup/restore	Evidencias firmadas por QA

Fase	Actividades	Criterio de cierre
P6 – entrega	Manual de usuario, capacitación, revisión académica	Acta de aceptación

5. Recursos y condiciones

Ambiente de desarrollo: .NET SDK 8, Node/npm, repositorios, motor DB compatible, conexión a sandbox Wompi, entorno Expo (SDK y dispositivo/emulador), herramientas de pruebas API y un lugar seguro para secretos. Host, proveedor, precio y SLA de producción quedan sujetos a decisión del equipo; no se asignan precios sin cotización actual.

6. Riesgos priorizados

Riesgo	Impacto	Respuesta
Duplicado de webhook	Alto	Idempotencia/firmas/eventos
Token o privilegios insuficientes	Alto	Pruebas OWASP API1/API5
Claves subidas a repo	Alto	Escaneo y rotación
Migración automática al iniciar	Medio/alto	Revisar estrategia en producción
CORS permisivo de desarrollo	Alto	Lista estricta de orígenes reales
Caída de SignalR	Medio	Reconexión + reconciliación por REST
Cobertura de pruebas desconocida	Alto	Construir pipeline CI

7. Criterios de éxito (a medir)

Operaciones funcionales críticas aprobadas, porcentaje de requisitos cubiertos por pruebas, incidencias bloqueantes resueltas, p95 de operaciones según carga acordada, trazabilidad de pagos y accesibilidad básica. Este plan no registra resultados cumplidos antes de ejecutar pruebas.

8. Plan de trabajo profesional

Fase	Actividad	Entregables	Puerta de aprobación
1	Levantamiento de alcance y criterios	Acta, actores, necesidades, matriz de requisitos	SRS aprobada, propietarios de requisitos definidos
2	Línea base de repositorios	Versiones Git, inventario de módulos y API	SHA de API/web/móvil, dependencias y riesgos
3	Cierre de arquitectura	Diagrama de componentes, ADR, elección de DB y despliegue	Diseño revisado por pares
4	Seguridad e integridad	Autorización por objeto, Wompi, migraciones	Pruebas negativas y control de pago
5	Pruebas integrales	E2E por actor, contrato, carga, accesibilidad	Informe de evidencias versionadas
6	Documentación y capacitación	Manuales, formación, soporte, transferencia	Aceptación registrada, incidentes abiertos controlados
7	Operación y mejora continua	Monitoreo, copias, auditoría, cambios	Indicadores, revisión y plan de respuesta

9. Matriz de recursos y presupuesto (sin valores inventados)

Concepto	Unidad de cálculo	Fuente para cotizar	Consideración
Servidor de API	vCPU/RAM y horas mensuales	proveedor VPS/PaaS seleccionado	incluye logs, TLS y reverso
Motor de datos	GB, conexiones, backups	servicio DB administrado o VPS	distinguir licencias SQL Server
Almacenamiento	GB y operaciones	servicio de objetos o volumen	fotos y documentos
Correo	mensajes/mes	SMTP transaccional	confirmaciones y recuperación
Pagos digitales	% comisión + costo fijo	condiciones vigentes Wompi	conciliación por pedido
Monitoreo	retención y volumen	proveedor telemetría	PII minimizada
Mano de obra	horas-persona por rol	tarifas del equipo	QA, desarrollo, soporte
Seguridad y copias	horas y GB	cotización y política	simulacros periódicos

10. Comparación de alternativas de implementación

Alternativa A: monolito modular existente. Ventajas: menor sobrecarga operativa y coherencia de transacciones en un solo backend; restricciones: crecimiento puede elevar acoplamiento. Es el patrón visible y el punto de partida recomendado para pruebas.

Alternativa B: microservicios desde el inicio. Ventajas potenciales: escalabilidad autónoma y despliegues separados; costos: red distribuida, observabilidad, contratos y consistencia eventual. No hay evidencia de que SaborExpressBack opere como conjunto de microservicios; no se recomienda describirlo así.

Alternativa C: backend gestionado de terceros. Puede simplificar alojamiento de recursos, pero implicaría rediseñar la API ya implementada. El README anterior describía Supabase y debe corregirse para mostrar el backend actual .NET.

Decisión provisional: estabilizar el monolito modular con pruebas, seguridad y documentación antes de reconsiderar límites de servicio.

11. Riesgos, responsables y condiciones de éxito

ID	Prob. cualitativa	Impacto	Mitigación	Responsable
R-01	Media	Alto	no subir secretos, monitorear cambios	Líder técnico (designar)
R-02	Alta	Alto	pruebas de firma/idempotencia en Wompi	Backend / QA (designar)
R-03	Media	Alto	prueba por motor EF Core	Backend/DBA (designar)
R-04	Alta	Medio	capacitaciones cortas por rol	Analista UX (designar)
R-05	Media	Alto	plan backup/restore	Operaciones (designar)

ID	Prob. cualitativa	Impacto	Mitigación	Responsable
			reproducible	
R-06	Media	Alto	aislamiento por sede y titular	Seguridad (designar)
R-07	Media	Medio	versionar contratos y cliente móvil	Frontend (designar)

12. Gestión de entregables y aceptación

Entregas incrementales: cada fase debe tener registro de decisión, enlace al commit, artefactos de revisión y aprobador. Sin datos de presupuesto o calendario acordado no se asignan fechas de ejecución ni compromisos económicos.

Definición de terminado (DoD): aceptación de alcance, pruebas positivas/negativas de funciones modificadas, cobertura de autorización sensible, manual actualizado, nota de versión, sin secretos, y reproducibilidad en ambiente limpio.

Análisis de retorno: calcular después de piloto con costo por pedido, tiempos, errores, cancelaciones y satisfacción, contrastados contra línea base. No afirmar ahorros que el estudio no ha medido.

10. Opciones de solución y evaluación comparada

Se consideran tres alternativas. (A) continuar con la aplicación API modular existente, reforzando pruebas, controles operativos y separación por componentes; (B) dividir dominios de negocio en microservicios independientes; y (C) reemplazar la capa de servicios por una plataforma BaaS de terceros. Las alternativas B y C implicarían migraciones de código, operación, contratos y persistencia que no están justificadas únicamente por el tamaño de la documentación. El análisis no pretende afirmar una selección oficial del equipo, sino ordenar las decisiones.

Criterio	A: API modular actual	B: microservicios	C: BaaS
Continuidad del código	Alta, coincide con repositorios	Baja/media; extracción de módulos	Baja; reescritura de reglas
Despliegue y observabilidad	Un servicio principal y dos clientes	Necesita tracing y redes de servicios	Depende de límites de proveedor
Integridad pedido–pago	Transacciones locales posibles	Requiere coordinación distribuida	Depende de APIs gestionadas
Costo operativo inicial	Relativamente contenido	Más alto por servicios/infraestructura	Variable por transacción/uso
Escalabilidad selectiva	Limitada por despliegue único	Granular por dominio	Acotada por proveedor

Criterio	A: API modular actual	B: microservicios	C: BaaS
Adecuación actual	Recomendación condicionada a QA	No se justifica sin cargas demostradas	No coincide con stack existente

11. Fundamentos de la arquitectura seleccionable

La API ASP.NET Core concentra contratos REST que consumen dos interfaces. Los controladores implementan entrada HTTP y delegan operaciones a módulos y servicios. Entity Framework Core configura el contexto y modelos relacionales; SignalR agrega un canal para notificaciones que no sustituye consultas de estado persistente. El módulo de pagos incorpora flujos de iniciación Wompi, recepción de webhook y conciliación. La coexistencia de tareas en segundo plano requiere distinguir su ciclo de vida de la respuesta HTTP de un pedido.

Las políticas de autenticación, autorización, turnos y visibilidad de sede se aplicarán en cada frontera de recurso. Un cliente que reciba un identificador no adquiere por ello facultad para consultarlo; la API debe comprobar propiedad o relación laboral explícita. El backend será fuente de verdad para importes, impuestos, estados de pedido y estado financiero, nunca el contenido modificable de una app.

12. Estrategia de implementación incremental

Iteración 1 – línea base: congelar el SHA de cada repositorio, documentar y compilar, corregir errores de configuración y controlar secretos. **Iteración 2 – contratos:** exportar OpenAPI, contrastar DTOs y mapear errores, incluir pruebas de autenticación, sedes y catálogo. **Iteración 3 – integridad:** cubrir pedido y sus detalles, transición de estados, Wompi sandbox y conciliación de pagos. **Iteración 4 – operación:** entregas, cocina, notificaciones y recuperación de conectividad. **Iteración 5 – liberación:** monitoreo, accesibilidad, respaldo y restauración. Cada iteración requiere criterios de aceptación observables antes de promover cambios.

13. Arquitectura de infraestructura propuesta

En desarrollo se permite ejecución local: .NET SDK para API, Node para web, Expo para móvil y un motor de base de datos elegido. En una eventual producción se propone red privada para la DB, proxy inverso HTTPS, terminación TLS, firewall, rotación de secretos, cuentas de servicio

de mínimos privilegios y copias cifradas. La selección de VPS/PaaS y sus costos no se presentan como decisión efectivamente ejecutada. Los precios de mercado varían y deben presupuestarse con cotizaciones vigentes, carga prevista y objetivos de recuperación establecidos.

El despliegue móvil se versionaría aparte, porque actualizaciones de clientes instalados pueden convivir con contratos viejos de API. Esa realidad exige ventanas de compatibilidad, cambios aditivos en DTO y un plan de versión mínima soportada. El front web puede liberarse de forma independiente siempre que la interfaz de servicios permanezca compatible.

14. Evaluación de seguridad y privacidad

Se recomienda matriz de amenazas que incluya elevación de privilegios, exposición entre sedes, manipulación del importe total, webhook suplantado, repetición del mismo pago, publicación de tokens y filtración de direcciones o documentos personales. Para cada amenaza deben definirse activo, actor, superficie de ataque, impacto, probabilidad razonada, mitigación, prueba y responsable. Los registros de operación deben conservar identificadores de correlación, no credenciales ni cuerpos completos con información sensible.

Se usarán OWASP API Security Top 10 (2023) y NIST SP 800-63B-4 (2025) como referentes; esto no significa certificación de cumplimiento. Las pruebas recomendadas incluyen ausencia de token, token expirado, empleado de otra sede, cliente ajeno, webhook de firma incorrecta, reintentos duplicados, denegación por turno inactivo y subida de imágenes con tipo inválido. El cumplimiento se consigna únicamente después de ejecutar controles y conservar evidencia.

15. Entregables verificables y criterios de aceptación

Entregable	Criterio verificable	Medio de comprobación
API documentada	OpenAPI coincide con build versionado	Export JSON + diff + QA
SRS	RF/RNF y matriz de aceptación aprobada	Acta y trazabilidad
Base de datos	Migraciones, DDL y relaciones reproducibles	Base efímera y respaldo
Pagos	Sandbox, checksum e idempotencia comprobados	Logs anonimizados y evidencia
Web	Rutas y roles críticos ejercitados	Capturas/test automatizado
Móvil	Flujos por perfil y red móvil ejercitados	Build + ensayo de dispositivo
Manual técnico	Permite ejecutar y diagnosticar	Revisión de operador
Investigación	Fuentes correctas y limitaciones explícitas	Revisión editorial

16. Presupuesto y riesgos de viabilidad

Los costos deben expresarse como intervalos con hipótesis explícitas: alojamiento de API, base de datos, dominio y TLS, almacenamiento y retención, correo/notificaciones, servicios externos, pruebas, soporte y mantenimiento. No se incluye un precio cerrado sin proveedor, moneda, plazo ni número de transacciones acordado. El presupuesto académico puede usar una hoja de estimación, separando CAPEX inicial de OPEX mensual; el costo de pasarela y mensajería se estimará mediante tarifa oficial vigente y escenario de volumen.

Los riesgos principales son dependencia de un integrador de pagos, cambios contractuales entre tres clientes, configuración inadecuada de CORS, recursos limitados para pruebas de regresión, seguridad de datos sensibles y falta de métricas operativas. Cada riesgo debe ligarse a una acción concreta y a un indicador de cierre, evitando afirmaciones vagas como “la solución es segura y escalable”.

Fuentes primarias de implementación

- API: https://github.com/yersonct/SaborExpressBack/tree/main
- Web: https://github.com/yersonct/SaborExpressWeb/tree/main
- Móvil: https://github.com/yersonct/SaborExpressMovil/tree/main
- Referencia de presentación: https://github.com/jsgh1/SRD-v1/tree/dev/docs/docs-SRD

Control documental: documento elaborado a partir del estado observado en octubre de 2026. Antes de entregar, verificar el commit exacto, responsable, revisión del instructor y cualquier cambio posterior del código.

17. Topología recomendada y decisiones justificadas

La arquitectura identificada es un **backend modular**; no hay evidencia de que sea una arquitectura de microservicios desplegados por separado. Mantenerla permite aprovechar controladores, módulos y contexto de EF Core sin una migración de alto costo. Se recomienda aislar las dependencias externas detrás de interfaces, practicar autorización por operación y entidad, centralizar contratos OpenAPI y efectuar migraciones de manera controlada.

Despliegue sugerido: cliente web Next.js, API .NET 8, motor relacional privado, terminación TLS, backup probado, logs centralizados y servicio Wompi externo. No se fija región, presupuesto ni proveedor sin cotizaciones y decisión formal. Para Node y Expo se recomienda una matriz de versiones derivada de package-lock y herramientas realmente instaladas.

18. Diagramas de soluciones

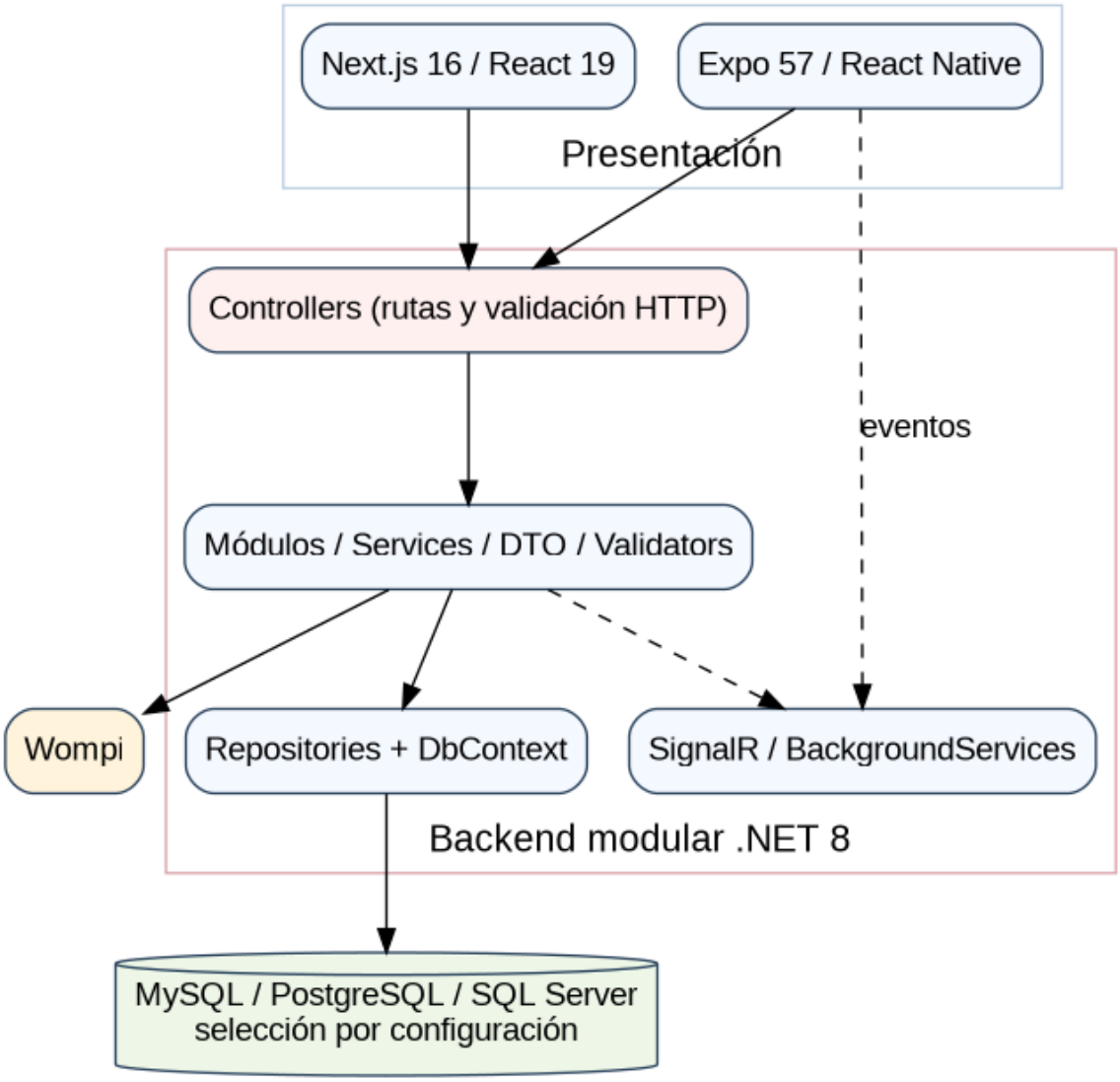


Figura 2. Componentes y responsabilidades

02_COMPONENTES

Nota. Elaboración para el estudio de caso SaborExpress; las figuras no certifican operaciones desplegadas.

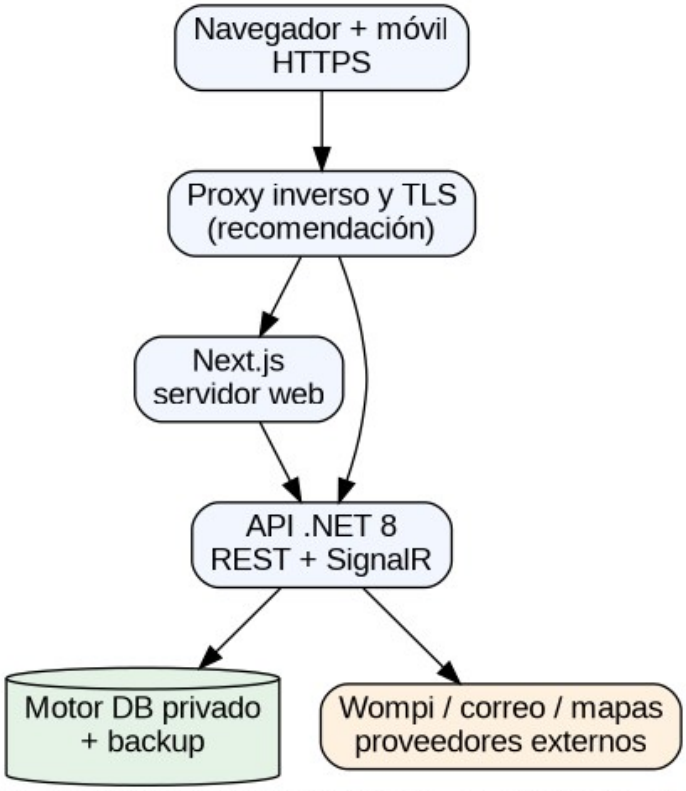


Figura 7. Despliegue lógico PROPUSTO: no es evidencia de hosting actual

07_DESPLIEGUE_PROPUESTO

Nota. Elaboración para el estudio de caso SaborExpress; las figuras no certifican operaciones desplegadas. ## 19. Criterios de adquisición y costo total de propiedad

Las ofertas de hosting se comparan por disponibilidad, costo de renovación, ancho de banda, soporte TLS, backups, residencia de datos, recursos de CPU/RAM, latencia, costo por transacción de pagos y soporte requerido. El costo total de propiedad incluye licencias si aplican, horas de ingeniería, monitoreo, incidentes, backups, dominio y cambios regulatorios. Las tarifas de proveedores fluctúan: una captura de precio no es presupuesto vigente.